Versatile Datapath Soft Error Detection on the Cheap for HPC Applications

Yafan Huang[†], Sheng Di[§], Zhaorui Zhang[‡], Xiaoyi Lu[¶], Guanpeng Li[†]
[†] Computer Science Department, University of Iowa, Iowa City, IA, USA
[§] Mathematics and Computer Science Division, Argonne National Laboratory, Lemont, IL, USA
[‡] Department of Computing, The Hong Kong Polytechnic University, Hong Kong
[¶] Department of Computer Science and Engineering, University of California, Merced, CA, USA
yafan-huang@uiowa.edu, sdi1@anl.gov, zhaorui.zhang@polyu.edu.hk, xiaoyi.lu@ucmerced.edu, guanpeng-li@uiowa.edu

***Abstract*—With the ongoing reduction in technology sizes and voltage levels, modern microprocessors are increasingly susceptible to soft errors, corrupting datapath units during program execution. While these error types have received considerable attention recently, existing solutions either confine themselves to limited scopes or incur massive overheads in performance and power consumption, hindering practical usage. In this work, we propose ConDa, a novel error detection technique based on code transformation and static program analysis, achieving versatile datapath protection at low cost. At compile time, ConDa analyzes program characteristics and transforms the original program code without complicating its control-flow and memory access patterns. At runtime, ConDa detects datapath errors with low overhead and latency. The evaluation of 38 benchmarks and a parallel HPC simulation reveals that ConDa only incurs 57.79% runtime overhead, which is 41.84% faster than existing state-of-the-art, with the same level of error detection effectiveness and low detection latency.**

***Keywords*—Reliability, Soft Errors, Datapath Protection, Code Transformation, Compiler, High-Performance Computing (HPC)**

I. Introduction

Due to the miniaturization of technology and the scaling of systems, transient hardware faults, also known as soft errors, have become increasingly common in modern high-performance computing (HPC) systems [1]–[5]. These faults typically originate in the lower levels of hardware components, propagate across layers during execution, and finally manifest at the program level, modifying program output and resulting in Silent Data Corruption (or SDC). Such failures have posed significant reliability challenges in today's HPC systems [6].

During a typical execution of an HPC program, millions of billions of instructions traverse through the datapath of processing units, including arithmetic execution components, registers, pipeline latches, and more. Consequently, there is a pressing need to shield these potentially vulnerable hardware components within the processor datapath to mitigate SDCs. Regrettably, efforts from both academia and industry to address this challenge have proven to impose unaffordably high overheads and costs in both performance [7] and energy consumption [8] in HPC systems.

In the past, research efforts such as EDDI [9], NZDC [10], MINPSID [11] etc [12]–[14] address register value corruption errors by employing instruction-level redundancy, while others like CFCSS [15], YACCA [16], CEDA [17], etc [18]–[20] focus on detecting errors that disrupt program instruction flow by tracking program execution patterns. While these techniques offer protection against faults occurring in specific components, their fault models often have a limited scope, rendering them incomplete for comprehensive protection of datapath. However, combining multiple protection techniques simultaneously results in significant overheads due to the inherent design differences of each individual technique [21], [22]. Consequently, HPC developers are reluctant to burden their systems with a heavy load of protection techniques, leaving SDC issues lingering in real-world HPC systems [7], [8]. Industry experience also corroborates the aforementioned concerns, as documented by Alibaba [23], Google [24], and Meta [25] in their respective datacenter operations.

In this paper, we tackle the above concerns by introducing a holistic soft error detection technique for the entire datapath, irrespective of the fault's origin within the datapath. In our investigation, we make two pivotal observations: (1) There exist distinct opportunities for leveraging the verification steps across different common detection mechanisms, owing to the repeated checking patterns. (2) The excessive runtime overheads in current detection techniques primarily stem from disruptions caused by amalgamating multiple detection methods in memory access patterns and program control-flow. Building upon these insights, we present a novel detection technique, ConDa, designed to replicate instructions with encoded execution flow. This protection safeguards various corruptions that lead to SDCs during program execution.

To elaborate further, ConDa first duplicates instructions of interest and encodes execution flow at basic block level, then introduces a thread-aware centralized buffer dedicated for temporary validation between duplicated copies as well as the encoded execution flow in order to minimize the possibilities that complicate control-flow. It seeks validation opportunities by facilitating compiler dominance within the program to preserve program memory access patterns. Finally, ConDa optimizes detection latency based on the locality of loops and branches and adaptively adjusts the timing of its operation throughout program execution. All of the steps are automated

This paper is accepted by SC'24.
Author's version is intended for personal use and not for distribution. The Definitive Version of Record is to appear at SC 2024.

in form of static analysis, resulting in negligible effort and time in executing CONDA workflow. We evaluate CONDA on 38 benchmarks and the major results are as follows:

- On average, CONDA has only 57.79% runtime overheads, which is 41.84% faster than state-of-the-art techniques.
- CONDA simplifies program control-flow by 2.64× and effectively preserves memory access patterns.
- CONDA achieves equivalent fault coverage effectiveness compared to EDDI and CFCSS, two leading software detection methods that focus on a single fault model.
- CONDA reduces the detection latency by 16.28× with only 4.65% additional runtime overhead, with the proposed adaptive detection placement algorithm.
- The design of CONDA is compatible with parallel programs, with 46.23% to 64.23% runtime overhead measured on a water reactor HPC simulation.

The rest of this work is structured as follows. In Section II, we present motivation, including scopes of existing works and challenges for holistic datapath protection. Then in Section III, we explain the problem in existing techniques and identify the root causes. Based on the root causes, we propose CONDA in Section IV and evaluate it in Section V along with a case study in Section VI. After presenting related works in Section VII, we conclude this work in Section VIII.

II. MOTIVATION

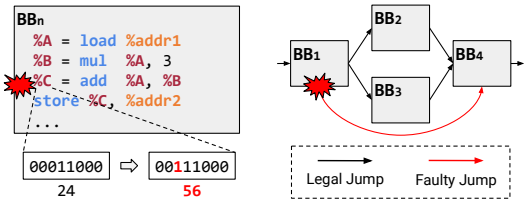
A. Existing Techniques and Their Scopes

Most existing error detection techniques for datapath units can be roughly classified into two fault models.

Detecting Erroneous Data: EDDI [9], SDCTune [12], SInRG [13], MINPSID [11], etc [10], [14], [26], [27] assume fault occurs at datapath components like load/store units, activating as corrupted register values at the instruction-level. To detect such erroneous values, these techniques duplicate the original instructions and compare the computation values in runtime. If values are mismatched, it means errors are detected.

Detecting Wrong Instructions: CFCSS [15], YACCA [16], CEDA [17], ACS [18], etc [19], [20], [28] assume error occurs at datapath units such as program counter, resulting in incorrect instructions to execute. These errors can be detected by inserting software signatures, which record legal execution footprint. In runtime, if the executed instruction is in unauthorized paths, then an error is detected.

Following our analysis, we find that all activated faults (e.g. errors that affect program execution) targeted by existing detection techniques manifest in two distinct patterns: *data-flow errors* and *control-flow errors*, which are illustrated in Figure 1. Such definition is also used in other closely related works [22], [28]. For data-flow errors, as seen in Figure 1(a), due to a bit-flip, the value in register *%C* is corrupted from 24 to 56, resulting in a wrong value stored in the address *%addr2*. A data-flow error can corrupt a value in any register, leading to wrong computation results at the end of a data dependency sequence (DDS)[1], possibly propagating to later program execution. Control-flow errors may activate in various ways, including corruption of the register holding target branch addresses, transforming an arithmetic instruction into a control-flow instruction, etc. However, all of these cases manifest the same issue – the incorrect target branch address (i.e. illegal jump), as shown in Figure 1(b). Note that large storage units such as cache can be protected by error correction code (ECC) [29] which is mature and shipped in most server-grade processors. Therefore, we do not consider faults that occurred in these components, which is a consensus across studies focused on datapath errors [11], [13], [18], [19].



(a) Data-flow error (b) Control-flow error

Fig. 1. Illustrating two distinct patterns of activated faults.

B. Holistic Datapath Protection and Challenges

Although existing error detection techniques are effective within their defined scopes, they assume that faults occur in specific, isolated locations, which is unreal in real-world scenarios. Faults can occur *anywhere* in datapath units, not just within single components like registers or program counters. A realistic approach should consider both data-flow and control-flow errors. As a result, to achieve complete datapath protection, it is necessary to combine these detection methods.

Some works, for example, SWIFT [21] and gZDC [22], combine the principle lies in EDDI [9] and CFCSS [15], targeting at complete protection. However, these methodologies bring new challenges – the required computations for these two techniques are highly different, resulting in excessive runtime overhead. Based on the report in [22], simply combining EDDI and CFCSS can incur as high as 400% runtime overhead (on average ∼200%). Such defect hinders them from practical usage in HPC. Reducing this overhead without losing any detection effectiveness is also the goal we want to achieve.

III. ROOT CAUSE ANALYSIS

In this section, we first perform an initial study and then understand the root causes behind such high overheads.

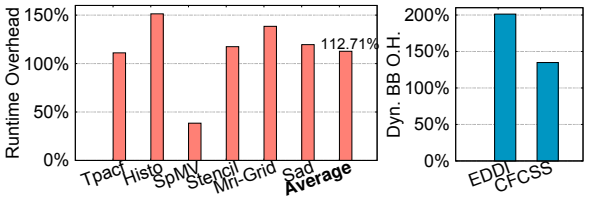
A. Initial Study

Recall that the main idea of current datapath detection methods, such as SWIFT and gZDC, involves integrating EDDI and CFCSS to identify errors in both data-flow and control-flow. We implement such a design and evaluate its performance.

[1]A DDS indicates a set of consecutive computation instructions where one instruction's return value may be operands of other instructions. State-of-the-art data-flow error detection [9], [11], [13] only checks once per DDS to reduce overhead. Note that one basic block may have multiple DDSs.

Note that our implementation of EDDI and CFCSS is inline with the state-of-the-art data-flow [11], [13], [26] and control-flow detection techniques [18], [19], [30]. Specifically, we select 6 benchmarks from Parboil benchmark suite [31] and quantify the runtime overhead via $RO_{pro}/RO_{ori} - 1$, where RO_{ori} and RO_{pro} denote the execution time of the original program and the program transformed with a protection technique. More detailed setups will be explained in Section V-A1.

Results can be found in Figure 2(a). **As we observe, the runtime overhead of state-of-the-art datapath detection techniques varies from 38.46% (on SpMV) to 151.34% (on Histo), with an average as high as 112.71% runtime overhead, which is equivalent to more than 2× execution time.** Such results demonstrate that although existing datapath detection techniques can guarantee the effectiveness of error detection, they sometimes result in excessive runtime overhead. This observation is consistent with existing works [22].



(a) Runtime overhead (b) Basic block overhead

Fig. 2. The problem and root causes of existing studies

B. Root Causes That Lead to High Runtime Overhead

What factors contribute to the substantial runtime overhead observed? To answer this research question, we perform a comprehensive ablation study for several possible factors, such as instruction workload, runtime basic block count, etc. We found there are two key defects in existing designs.

> **Defect 1:** Existing detection techniques introduce extra branches for reporting detection results, which significantly complicates the original program control-flow, negatively impacting the runtime performance.

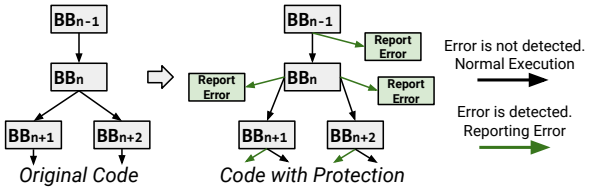


Fig. 3. Illustrating why error detection techniques (both data-flow and control-flow) can complicate original program control-flow.

To decide whether an activated fault is detected, existing works (e.g. EDDI and CFCSS) save the checking results (`TRUE` or `FALSE`) in a new register. Different from the original program continuing to normal execution, this register will point at two different branches (i.e. basic blocks), indicating whether an error is detected or not (see Figure 3). In data-flow detection methods, the state-of-the-art works insert one extra branch for every DDS, which even breaks one basic block multiple times. We profiled such results in Figure 2(b). As we can see, EDDI and CFCSS lead to 201.23% and 134.84% dynamic basic block overhead (i.e. extra basic block executed in the runtime), drastically complicating the original program control-flow, diminishing the efficacy of downstream compiler optimizations, reducing runtime performance in return.

> **Defect 2:** Some detection techniques require frequent memory accesses for error checking, which may corrupt coalescing memory access patterns in original programs, leading to higher runtime overheads.

For mitigating the first defect, existing work [13], [32] buffer checking results temporarily and only introduces a detection branch once per function. This approach necessitates frequent access to global memory (more details will be explained in Section IV and Figure 7(a)), and such introduced memory operations may lead to strided memory-accessing behaviors and hence reduce performance in return. This phenomenon is more obvious in programs with rigid and systematic memory access patterns, such as Parboil SpMV (sparse matrix-vector multiplication). In each iteration of SpMV, the buffering of results interrupts the sequential data access to the target matrix, breaking the expected coalescing and resulting in a performance slowdown of over 60%, even when the control-flow remains simplified. Note that control-flow detection (including CFCSS) also has memory operations for storing constant to global variables (updating software signatures to record execution footprint). However, our findings indicate that such instructions incur a relatively lower computational overhead and compiler itself also has a strong constant folding and propagation algorithm, making this operation not a bottleneck.

Based on these findings, we propose ConDa, a compiler-level co-design for versatile datapath detection with low cost. We will further explain ConDa designs in Section IV.

IV. ConDa: A Fast and Versatile Detection Technique for Datapath

In this section, we first introduce ConDa from a high-level overview and then explain each major step in detail.

A. High-level Overview of ConDa

ConDa is a fast error detection technique for datapath units. The key insight of ConDa is that we found the excessive runtime overhead of existing datapath detection techniques arises from complicated program control-flow (examples see Figure 3) and extra global memory accesses. Building on these insights, we propose ConDa, of which a high-level overview is shown in Figure 4. Given the program source code in LLVM IR (intermediate representation) format, ConDa can generate the protected program source code (still in LLVM IR) based on 3 major steps. **Lazy Checking Motion**

(❶): Compared with existing works that combine EDDI and CFCSS straightforwardly [21], [22], CONDA performs code transformation for instruction-level redundancy and software signature via a lazy checking motion mechanism, buffering error-checking results for all DDSs and branch jumps for each basic block temporarily within a single register for later usage. By doing so, CONDA does not introduce extra basis blocks and preserves the original program control-flow. This step resolves Defect 1 in Section III-B. **Domination Propagation (❷)**: After analyzing the domination tree of a function, CONDA aggregates the temporarily buffered error-checking results for basic blocks as the domination relationship propagates. This allows CONDA to synchronize across basic blocks without accessing memory, reducing overhead compared with existing aggregation method [13]. This step resolves Defect 2. **Adaptive Detection Placement (❸)**: Finally, CONDA places detection instructions based on an adaptive algorithm to generate the CONDA-protected code. This step is based on static program analysis for each function and targets achieving full fault coverage for the aggregated error-checking results without introducing too much detection latency. Note that the overall workflow of CONDA does not require any program executions and can be fully automated by a single compiler-level pass, so the workflow runtime is almost negligible.

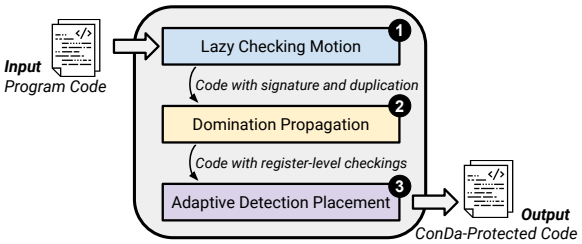


Fig. 4. High-level overview of CONDA workflow

B. Design Details of CONDA

In this section, we elaborate on the technical details of the three major steps outlined in Figure 4.

1) Lazy Checking Motion (❶): This step postpones the checking mechanism to later steps and inserts duplicated instructions and signature computations without modifying the program control-flow. Given the program's original code in each basic block, CONDA first analyzes its control-flow and data-flow, identifying all legal predecessors[2] and DDS. Then, CONDA inserts duplicated instructions for each DDS and software signatures for each corresponding predecessor basic block. Instead of adding detection instructions directly, CONDA buffers comparison results in a set of registers (*TRUE* for correct execution) and performs *AND* operation for each two of them. Supposing the comparison results are temporarily saved in n registers in one basic block, there will be $n-1$ *AND* operations introduced, where the return register of the last *AND* saves the checking results of the whole basic block. The register in the final *AND* operation can hold a *TRUE* value only if all preceding registers contain *TRUE* values, indicating an error-free execution in this basic block. Note that lazy checking motion only buffers error-checking results, and CONDA will aggregate these buffered results and add the corresponding detection strategically in the latter two steps.

[2]A *predecessor* of a basic block is any basic block that can directly transfer control to it. In contrast, a *successor* of a basic block is any basic block to which control can be directly transferred from it.

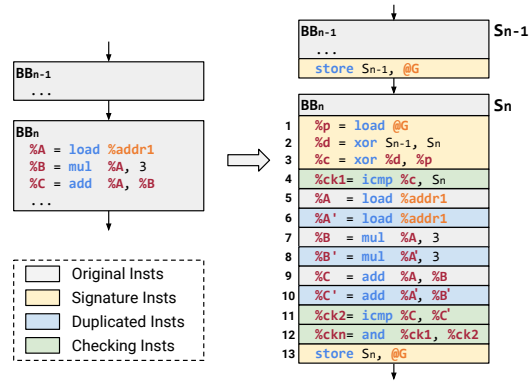


Fig. 5. Transforming original code with lazy checking motion mechanism. The inserted signature and duplicated instructions are used to detect control-flow and data-flow errors, respectively.

Figure 5 illustrates an example of lazy checking motion. As seen, BB_n has one legal predecessor BB_{n-1} and one DDS (composed by 3 instructions). After the signature and duplicated instructions are inserted, CONDA buffers the control-flow and data-flow checking results in register *%ck1* and *%ck2*. If the values of *%C* and *%C'* remain consistent and the branch jump is legal, *%ck1* and *%ck2* will hold *TRUE* values. Finally, an *AND* operation is executed between these two registers, with the result stored in a new register *%ckn*, guarding BB_n against datapath errors. We will use *%ckn* to denote the whole buffered checking results for BB_n in later contexts.

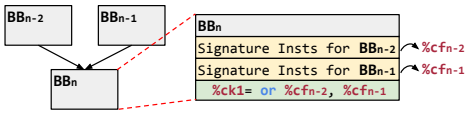


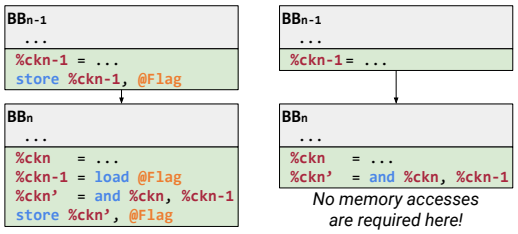
Fig. 6. Fan-in control-flow checking in lazy checking motion

For control-flow checking in fan-in scenarios (i.e. one basic block has multiple predecessors), lazy code motion is illustrated in Figure 6. Here, BB_n has two legal predecessors BB_{n-1} and BB_{n-2}. In this case, CONDA first inserts signature instructions for each of the legal predecessors, saving the comparison results as *%cfn-1* and *%cfn-2*. Then, an *OR* operation is performed between these two results, saving the result in *%ck1*. By doing so, as long as the previously executed basic block is one of the legal predecessors, *%ck1* can maintain a *TRUE* value, allowing the program to continue normal execution. In our implementation, we also consider inter-procedural jumps, which are function calls from other basic blocks outside the current function, as legal predecessors. Thus, CONDA can detect both intra- and inter-procedural control-flow errors. It is true that this design cannot detect legal but faulty control-flow errors. However, we argue that this is a common flaw

for all signature-based branch-checking mechanisms, such as other state-of-the-art control-flow detection algorithms in this literature [15], [18], [19]. This drawback can be covered by CONDA with data-flow protection.

CONDA is also compatible with OpenMP multi-threaded programs. In LLVM IR, an OpenMP flag `#pragma omp` will be treated as a separate function (we define it as an *OpenMP kernel*). For control-flow checking, if multiple threads update the execution footprint into the flag *G* (in Figure 5), such race condition may trigger a false report. However, CONDA resolves this via a thread-preserve flag. For rest functionalities, lazy checking motion buffers error-checking results within each OpenMP kernel in the same way.

2) Domination Propagation (❷): This step aims to aggregate buffered checking results across basic blocks inside a function without time-consuming memory operations – making all aggregation within register level, hence improving the runtime performance of CONDA. The necessity for aggregation is explained as follows. In the lazy code motion step, CONDA buffers error-checking functionalities of one basic block into a single register. If directly adding detection instructions here (example see Figure 3), one extra branch will still generate for each static basic block, highly complicating control-flow, reducing performance in runtime. In contrast, if such registers are aggregated within a function, all basic blocks in this function can share one detection branch, effectively preserving the program's original control-flow.



(a) Using global variable (b) Using register

Fig. 7. Aggregating checking results across basic blocks (left using global variables, right using registers). The green regions are checking instructions.

In the past, synchronizing across basic blocks was routinely conducted via memory accesses, such as load/store operations from a global variable. Mahmoud et al. [13] proposes an EDDI-based data-flow error detection technique that accumulates local checking results using memory. The basic idea is shown in Figure 7(a). Given basic blocks BB_{n-1} and BB_n, the checking results for each are buffered in *%ckn-1* and *%ckn*. In BB_n, *%ckn-1* is first loaded from a global variable *@Flag*. To aggregate these two values, an *AND* operation is conducted here, making sure both basic blocks execute correctly (i.e. value as *TRUE*). After the aggregation, the generated *%ckn'* is stored back to global variable *@Flag* for later usage. In this case, a pair of load-store operations is needed in each static basic block. During program execution, this may lead to billions of extra global memory accesses, causing pipeline stalls or strided memory access patterns [33], negatively impacting the runtime performance. Figure 7(b) illustrates a promising solution, which aggregates values from registers directly. However, employing registers in instruction code presents considerable challenges due to the strict lifetime constraints they possess compared to source code levels and the necessity to adhere to the Single Static Assignment (SSA) format. To resolve this, domination propagation is proposed in CONDA and it is mainly conducted by domination analysis.

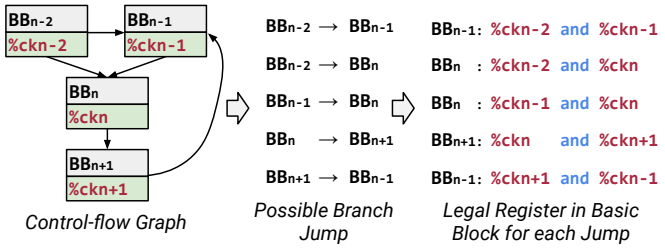


Fig. 8. Domination analysis for a program control-flow. Green region includes the checking result for a basic block.

Figure 8 shows an example of domination analysis. Domination analysis guarantees current basic block only aggregates the register value from its dominating 1-hop successor, thereby avoiding memory accesses. Given the control-flow graph of a function, CONDA first identifies all possible branch jumps, and each of them is formulated by "source basic block" → "destination basic block". For example, in jump $BB_{n-1} \rightarrow BB_n$, BB_n aggregates its own buffered checking results *%ckn* along with *%ckn-1*. By conducting this, CONDA legally aggregates all buffered registers in a function through the propagation of the domination tree. For the fan-in cases, one basic block may have multiple successors and aggregating which register requires a selection. We implement such a selection using a *PHI* instruction in LLVM IR. One example is illustrated in Figure 9. In this *PHI* node, if the previously executed basic block is BB_{n-1}, it will select the value from the register *%ckn-1*, otherwise *%ckn-2*. In x86 instruction set architectures (x86 ISA), such an instruction is compiled into the right part in Figure 9. The selection between *%ebx* and *%ecx* registers, which contain the results of *%ckn-1* and *%ckn-2*, is decided by the results of the comparison instruction. Indeed, a control-flow instruction is introduced here. However, we reckon that this is only a short jump on one or two assembly instructions, which routinely have more compact encoding, simpler address calculation, and higher cache efficiency, making it faster than long jumps such as basic block levels.

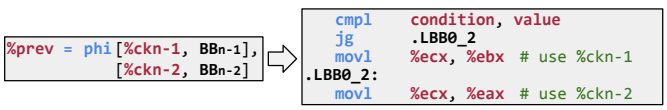
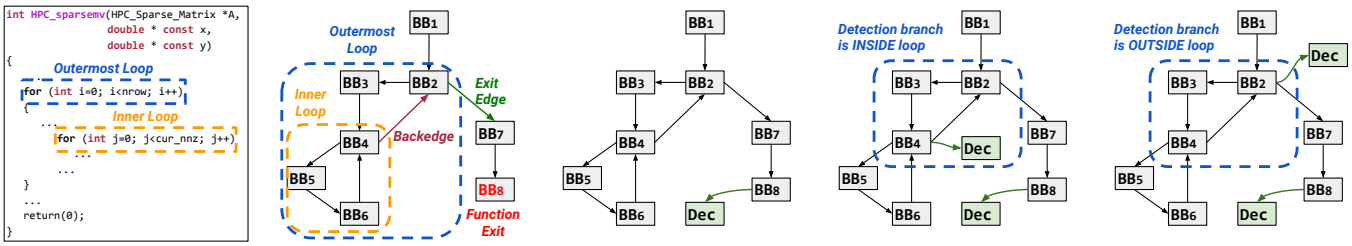


Fig. 9. Fan-in register selection in LLVM IR (left) and x86 ISA (right)

3) Adaptive Detection Placement (❸): Given each function in the program, this step classifies them into 1 of 3 cases and places detection branches accordingly, achieving low detection latency and full error detection effectiveness. This step also applies to OpenMP kernels. Recall that in steps ❶



(a) Source Code (b) Control-flow graph (c) Case-1: $N = 0$ (d) Case-2: $N = 1$ (e) Case-3: $N > 1$

Fig. 10. Illustrations of CONDA Adaptive Detection Placement. **Dec** indicates a detection branch (i.e. basic block) to be placed. We use *HPC_sparsemv* function from Mantevo HPCCG to explain all three cases. In CONDA, there is only one outermost loop in this function, so Case-2 is selected in this example.

and ❷, CONDA buffers the checking results and aggregates them along with function control-flow. A detection branch placed at the function exit suffices to report any data-flow or control-flow error detected within the function. Note that the detection branch is an extra basic block that reports errors, which has the same functionality as the *Report Error* basic block mentioned in Figure 3. However, such a placement can lead to a new problem – long detection latency. Compared with most of the existing works [9], [11], [15], [27] that directly report errors at the end of each DDS and basic block, this placement can only report errors when the function terminates, no matter where the error occurs. In HPC simulation, long latency requires more time or iterations to trigger recovery mechanisms, such as checkpoint restart [34], leading to a huge waste of performance and energy. It also complicates bug localization for performing downstream protections such as instruction-level patches [14], posing substantial challenges to constructing reliable systems. This defect motivates us to place detection branches strategically in CONDA.

Algorithm 1 CONDA Adaptive Detection Placement

Input: Function after CONDA ❶ and ❷ transformations.
Output: Function with detection branches placed.
1: Generate the control-flow graph *CFG* of this function.
2: Place detection at function exit. *// Case-1*
3: Calculate the number of outermost loops in *CFG* as *N*.
4: **if** $N > 1$ **then**
5: Place detection at exit edge of each outermost loop. *// Case-2*
6: **end if**
7: **if** $N = 1$ **then**
8: Place detection at backedge of the outermost loop. *// Case-3*
9: **end if**

Algorithm 1 presents the adaptive detection placement (❸) in CONDA for each program function. Give a function transformed by ❶ and ❷, CONDA first generates its control-flow graph, which is the same as that of the original program since the first two steps of CONDA only perform operations at the instruction level and hence preserve basic block structures. Then, CONDA places a detection branch at the function exits, to ensure all error-checking results are covered. After that, CONDA performs loop detection to count the number (denoted as N) of the outermost loop (i.e. highest-level loop) in this function. If $N > 1$, CONDA places extra detection branches at the exit edge of each outermost loop. If $N = 1$, CONDA places extra detection branches at the backedge of the outermost loop. By doing so, CONDA effectively reduces the detection latency without introducing a high control-flow penalty in the runtime.

Figure 10 further explains the 3 cases of this adaptive design with a running example. We use *HPC_sparsemv* function from Mantevo HPCCG for illustration. The function source code can be seen in Figure 10(a). Figure 10(b) presents the function control-flow graph and corresponding loop terminologies. Here, the backedge indicates the edge that executes back to the header basic block of the outermost loop, whereas the exit edge means the edge that jumps out of the outermost loop. When $N = 0$ (Figure 10(c)), no loop is in the current function. Such functions usually have simple structures and can finish very fast, and CONDA only places a detection branch at the exit nodes without incurring so much latency. When $N = 1$ (Figure 10(d)), this function has only one outermost loop. In this case, beyond the detection at the exits, CONDA also places a checker at the loop backedge, which is included in each loop step, to reduce latency. The incurred runtime overhead remains manageable as the most computation-intensive inner loop maintains its original control-flow. When $N > 1$ (Figure 10(e)), there may be multiple outermost loops and the function structure tends to be very complex. Thus, CONDA places all detection branches outside computation-intensive loops, which includes each loop exit and the function exits. This reduces latency while still controlling the control-flow complexity. Indeed, adaptive detection placement is a heuristic approach and may not be the optimal solution for balancing low detect latency and low control-flow complexity. However, localizing the optimal solution is non-trivial and may require dynamic profiling, which is extremely expensive for HPC scenarios. In CONDA, this step only requires static program analysis, so the overhead remains almost negligible.

V. EVALUATION

We first introduce the experimental setups, including the experimental platform, benchmark selection, and evaluation methodology. Then we present evaluation results for CONDA, prioritizing runtime overhead as the most crucial metric, while also considering fault coverage and detection latency.

A. Experimental Setups

1) Platform: We conduct experiments on a 15-node local cluster, where each node has an Intel Xeon Gold 5218R CPU and 32 GB memory. The operating system is Ubuntu 20.04

and all benchmarks are pre-compiled to LLVM IR via -O0[3] optimization under LLVM V15.0, which is also the version that we implement CONDA and all other baseline methods.

TABLE I
38 BENCHMARKS SELECTED FOR EVALUATING CONDA

Benchmark	Suite	Benchmark	Suite
CoMD	*Mantevo*	CRC32	*MiBench*
HPCCG	*Mantevo*	BasicMath	*MiBench*
XsBench	*CESAR*	BitCount	*MiBench*
SP	*NPB*	PathFinder	*Rodinia*
BT	*NPB*	KNN	*Rodinia*
IS	*NPB*	Needle	*Rodinia*
UA	*NPB*	LUD	*Rodinia*
CG	*NPB*	BFS	*Rodinia*
LU	*NPB*	BackProp	*Rodinia*
DC	*NPB*	KMeans	*Rodinia*
MG	*NPB*	ParticleFilter	*Rodinia*
EP	*NPB*	Ocean	*SPLASH-2*
Dijkstra	*MiBench*	FFT-K	*SPLASH-2*
StringSearch	*MiBench*	Tpacf	*Parboil*
iFFT	*MiBench*	Histo	*Parboil*
FFT	*MiBench*	SpMV	*Parboil*
Patricia	*MiBench*	Stencil	*Parboil*
QSort	*MiBench*	Mri-Grid	*Parboil*
Susan	*MiBench*	Sad	*Parboil*

2) Benchmark Selection: We try to include as many benchmarks as possible for a fair and comprehensive evaluation of CONDA. To this end, we select 38 benchmarks from 7 benchmark suites [31], [35]–[40] from various domains (see Table I). We rename FFT from SPLASH-2 to FFT-K to distinguish it from the one in MiBench. These benchmarks include almost all benchmarks that are selected in recent error resilience studies [10], [11], [27], [30], [41]–[46].

3) Evaluation Methodology: As mentioned in Section IV, we evaluate CONDA from three perspectives, which are runtime overhead, fault coverage, and detection latency. For measuring program *runtime overhead*, we execute each program 5 times and obtain its average. This is also the key metric of CONDA design. The way we calculate its corresponding overhead can be found in Section III-A. Note that all the programs are executed with test input provided by the benchmark suite. For *fault coverage*, we use SDC reduction, since existing detection techniques [11], [18], [19], [27] assume they can be seamlessly integrated with other post-recovery mechanisms such as checkpoint/restart [34], making other failure outcomes such as crash less harmful. For *detection latency*, we measure the number of dynamically executed instructions between the locations when the error occurs and is detected.

For baseline datapath protection, we adopt the same design from SWIFT [21] and gZDC [22] and implement it in our platform[4]. For simplicity, we record this name as *Baseline*. For single data-flow or control-flow detection, we adopt *EDDI* (instruction-level redundancy) and *CFCSS* (software signature). Although we use the names from [9] and [15], our implementation of EDDI and CFCSS is inline with the state-of-the-art approaches [11], [13], [18], [19], [27], [30].

[3]We also run our techniques with different optimization levels (e.g. -O2), and the conclusion for runtime overhead in this work remains the same.

[4]In [21] and [22], SWIFT and gZDC were proposed and implemented using OpenIMPACT [47] and LLVM-or1k [48] compiler frameworks, respectively. Besides, [21], [22], and this work are evaluated under different hardware and software settings. These explain why baseline runtime overheads reported in this work are higher than [21] and lower than [22]. However, we unify code transformation for all methods under LLVM V15.0 and measure their runtime overheads with the same settings, for a fair comparison.

B. Runtime Overhead

Figure 11 presents the results of runtime overhead for CONDA on 38 selected benchmarks. For comparison, we use *Baseline* and CONDA-*mem*, two complete datapath error detection techniques. While Baseline is explained in Section V-A3, CONDA-mem here denotes the CONDA code transformation without *Domination Propagation* (❷) – it uses global variables and memory operations to synchronize error-checking results across basic blocks along with ❶ and ❸ (see Figure 7(a)). We use CONDA-mem as a baseline to demonstrate the superiority in register-level computations. Note that runtime overhead is the key metric to evaluate CONDA.

On average, CONDA reduces the runtime overhead by 41.84% compared with Baseline detection method, and **it only incurs 57.79% runtime overhead on 38 benchmarks**, with the highest overhead of 199.97% in UA benchmark. In contrast, Baseline can lead to 99.38% runtime overhead, varying from 1.87% in BFS benchmark to 242.83% in UA benchmark. With the simplified control-flow complexity, CONDA-mem achieves higher performance compared with Baseline, with 81.54% runtime overhead on average. However, it is 29.12% slower due to the introduced memory operations, compared with CONDA. Besides, there are three important observations, which will be explained as follows.

(Observation 1)*: Despite the average, the runtime overheads of* CONDA*,* CONDA-*mem, and Baseline are highly application-specific, but* CONDA *always have the lowest overhead.* For example, in CONDA, the highest and lowest runtime overheads are observed in UA and FFT benchmarks, respectively, with a substantial difference of 212.70% between them. Such differences are 248.31% and 240.96% in CONDA-mem and Baseline. Many program characteristics can contribute to such a large variance simultaneously, such as its intrinsic natures (e.g. arithmetic-heavy or logic-heavy), data intensity, memory/cache utilization, etc. However, CONDA consistently outperforms CONDA-mem and Baseline, achieving the lowest overhead on 36/38 benchmarks. This demonstrates the effectiveness of our identified defects from Section III-B.

(Observation 2)*: In benchmarks such as StringSearch and Patricia, the runtime overheads are negative numbers, indicating sometimes program execution with* CONDA *is even faster than that of the original program.* For example, in StringSearch, FFT, and Patricia benchmarks, the runtime overheads are just -10.26%, -12.73%, and -11.36% while using CONDA protection. We find that the root cause is that some benchmarks underutilize hardware recourses, leading to lower runtime performance. Additionally, in very few benchmarks, the CONDA may unexpectedly improve memory behaviors. In StringSearch, all candidate strings with various lengths are stored as global variables, and loading them from memory sometimes leads to irregular access patterns. In CONDA, the load instructions are duplicated, aligning memory accesses more sequentially, increasing the use of each cache line,

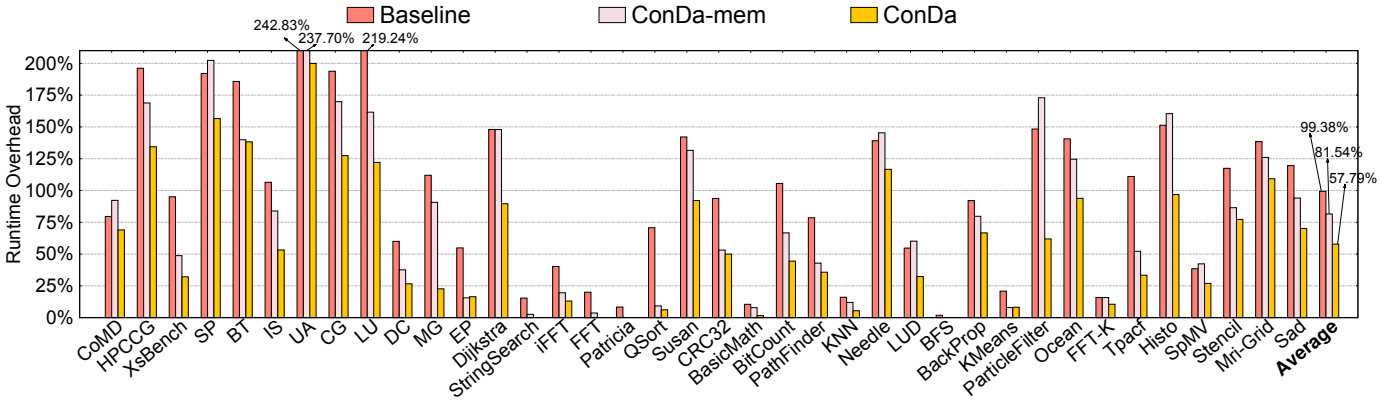
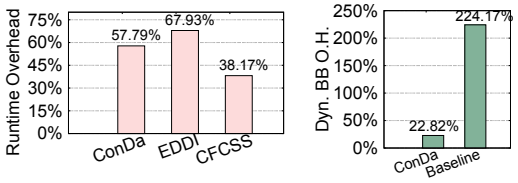


Fig. 11. Runtime overhead evaluation. Some bars (e.g. Patricia in ConDa) are not shown due to negative overhead, which is explained in Section V-B. *Baseline* here denotes the same design with existing datapath detection techniques including SWIFT and gZDC.

and enhancing spatial locality in return. While this negative overhead is also observed in ConDa-mem, Baseline, with complicated control-flow, can hardly lead to a faster program execution – the lowest overhead is only 1.87% on BFS. This observation further demonstrates a key design in ConDa– simplifying program control-flow. Note that this negative overhead is also observed in recent fault tolerance works [13], [32].

(Observation 3)*: In benchmarks such as SpMV, ParticleFilter, and SP, even* ConDa*-mem simplifies the program control-flow, it still leads to higher runtime overhead compared with Baseline.* In SpMV and ParticleFilter, the runtime overheads of ConDa-mem are 42.31% and 172.94%, whereas these two numbers are 38.46% and 148.36% in Baseline. We investigate these benchmarks and find that they all require rigid memory access patterns. For example, SpMV is a benchmark that evaluates sparse matrix-vector multiplication, ParticleFilter requires massive matrix computations for processing video frames and calculating their spatial localities, and SP is designed to solve a block penta-diagonal system. In these cases, the strided memory access patterns can cause a higher performance penalty while optimizing the program control-flow. While using ConDa, due to the register-level aggregation in ❷, the original memory behavior is highly preserved while adding datapath protection. Still taking ParticleFilter as an example, the runtime overhead 172.94% in ConDa-mem is drastically reduced to 61.92% in ConDa. This result demonstrates memory optimization in ConDa design (❷).



(a) Ablation study for Baseline (b) Basic block overhead

Fig. 12. Other factors related to runtime performance evaluation.

To further understand the runtime overhead in ConDa and baseline methods, we perform an ablation study for Baseline. and results are in Figure 12(a). Note that this is not a strict ablation study, since the baseline also combines EDDI and CFCSS inside basic blocks strategically (to be consistent with SWIFT and gZDC), making it very challenging to measure the instruction-level performance gain precisely. As a result, we measure the runtime performance for EDDI and CFCSS separately on an average of 38 selected benchmarks. There are two important takeaways. (1) In ConDa, the runtime performance penalty from data-flow protection is usually higher than that from control-flow protection. (2) With the proposed optimization, ConDa even has lower runtime overhead compared with EDDI, which only provides data-flow error detection.

Similar to Section III-B, we measure the number of dynamically executed basic blocks to quantify the ability in ConDa to simplify the program control-flow. We present the results in Figure 12(b). In Baseline, the average dynamic basic block overhead is 224.17%. This number even exceeds 500% in some benchmarks such as BT, where one basic block can have as many as 483 instructions, composing more than 10 DDS, hence breaking the program control-flow multiple times in the baseline method. However, the average basic block overhead is reduced by 8× (22.82%) in ConDa, proving the effectiveness in lazy checking motion (❶).

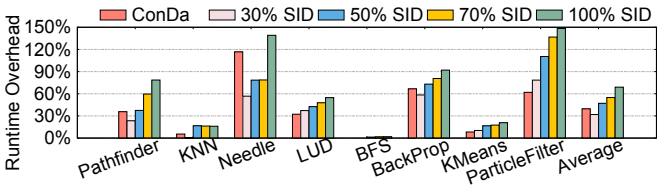


Fig. 13. Compare ConDa with selective instruction duplication.

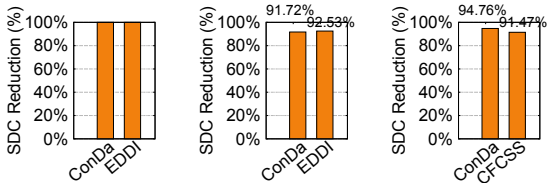
We also compare runtime overhead between ConDa and selective instruction duplication (SID) on 8 Rodinia benchmarks. Results are in Figure 13. SID selectively duplicates the most vulnerable code regions and has been studied recently [11], [12], [14], [27]. "30% SID" denotes detecting as many silent faults as possible within 30% runtime overhead budget. Note that the runtime overhead here is measured by the dynamic instruction cycle (the same design with [11] and [14]). Although we maintain the name "SID", we include control-flow detection in this method. With the performance

optimization, CONDA has lower overheads compared with 50% SID (39.59% vs 47.07%), where only 50% overhead budget is allowed for SID, whereas CONDA achieves full detection at the same level.

Highlight 1: On average, CONDA achieves a runtime overhead of only 57.79%, outperforming the baseline method by 41.84%, attributable to its preservation of program control-flow and memory access patterns.

C. Fault Coverage

To evaluate CONDA's ability to detect datapath errors, we measure the fault coverage by performing fault injection experiments and examining the SDC reduction. For data-flow errors, the transient hardware faults are emulated by randomly selecting a dynamically executed instruction in runtime and flipping a single bit within the destination register. We inject faults at both LLVM and assembly levels, as they are demonstrated both effective in studying error resilience [49]. While LLVM-level fault injection is implemented by compiler code transformation, we perform assembly-level fault injection using a dynamic instrumental tool Intel PIN [50]. The fault injection methods we use are consistent with other LLVM-level [43], [51], [52] and assembly-level fault injectors [53], [54] in this literature. For studying control-flow errors, same as [18], [19], we randomly select an executed control-flow instruction during program runtime and distort its target address into an erroneous one. While corrupting control-flow in LLVM IR is not enough to simulate such a low-level behavior, we conduct control-flow fault injection on assembly-level using Intel PIN. This aims to emulate hardware faults that affect control-flow-related datapath units such as program counters. The way we perform fault injection for studying control-flow errors is inline with other state-of-the-art works using architectural simulators [18]–[20]. For both control-flow and data-flow fault injections, we assume there is only one fault per program execution – this is a common agreement in soft error resilience works [18], [55]–[57].



(a) LLVM-level data-flow FI (b) Assembly-level data-flow FI (c) Assembly-level control-flow FI

Fig. 14. Fault coverage evaluation, where FI denotes "fault injection". These results are averaged across 38 selected benchmarks.

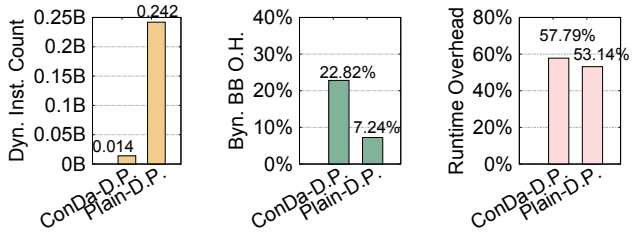
Figure 14(a) and 14(b) presents the fault coverage for data-flow errors. For LLVM- and assembly-level fault injections, we perform 1,000 fault injections for each benchmark, calculate the SDC reduction (measured in %), and report the average on all selected benchmarks. We select the baseline method as EDDI, and the goal of CONDA is to provide similar protection effectiveness, demonstrating the runtime overhead gain from CONDA is not a trade-off. At LLVM-level, both CONDA and EDDI can achieve 100% SDC reduction, since both CONDA and EDDI are implemented via LLVM transformation passes. For assembly-level fault injections, we exclude some cross-layer deficiencies since they can be simply resolved by LLVM-level patches [14]. As seen, CONDA and EDDI have 91.72% and 92.53% SDC reductions. The SDC reduction of CONDA is highly stable across benchmarks, varying from 85.44% to near 100%. As a result, CONDA has almost identical data-flow error detection ability compared with EDDI.

Figure 14(c) shows the results of control-flow errors. We use CFCSS as the baseline method and perform 1,000 fault injections for each benchmark. Surprisingly, CONDA achieves slightly higher SDC reduction compared with CFCSS (94.76% vs 91.47%). The reason is that the data-flow detection in CONDA can detect control-flow errors in certain cases, such as when illegal jumps corrupt the register in arithmetic instructions and intra-block illegal jumps. In all, CONDA maintains the same level of detection ability for control-flow errors.

Highlight 2: While successfully reducing the runtime overhead, CONDA also maintains the same level of error detection ability compared with EDDI and CFCSS.

D. Detection Latency

We specifically evaluate the detection latency for the adaptive detection placement (❸) algorithm in CONDA. To establish a benchmark for comparison, we use a plain detection placement, which indicates only placing detection branches at the function exits in CONDA code transformation (i.e. treating all functions like the Case-1 in Figure 10(c)). This design is utilized in existing works [13], [32] that try to simplify control-flow without losing detection effectiveness. Due to fewer detection branches added, this plain detection placement has a high detection latency but also with an even more simplified program control-flow than CONDA itself – thus we evaluate both pros and cons here.



(a) Detection latency (b) Basic block overhead (c) Runtime overhead

Fig. 15. Detection latency evaluation of CONDA, where "D.P." denotes "detection placement" and "plain" indicates the algorithm that only places detection branches at the function exits (the same design in [13]).

For the pros, Figure 15(a) shows the results of detection latency between CONDA and the baseline methods on an

average of 38 benchmarks. Since the plain detection placement can lead to as high as 0.242 billion of dynamic instructions, CONDA reduces this number by 16.28×, mitigating the detection latency brought by lazy checking motion (❶). Despite the latency, our analysis shows that CONDA significantly reduces the number of static basic blocks by over 4×. This reduction greatly aids in localizing the activated faults.

For the cons, since adaptive detection placement (❸) introduces extra detection branches, the baseline can have a simpler control-flow and possibly lower runtime overhead. We measure them in Figure 15(b) and 15(c). As seen, although CONDA introduces 15.58% dynamically executed basic blocks, it only incurs an extra 4.65% runtime overhead on selected benchmarks. One corner case is HPCCG benchmark – we find more than 50% runtime is contributed by *HPC_sparsemv* function, which only contains a simple nested loop. In that case, CONDA places the detection branch at the backedge of the outermost loop, which is included in loop execution (see Figure 10(d)), leading to higher runtime overheads. However, based on results from the rest benchmarks, we reckon that CONDA always leads to negligible runtime overhead (less than 5%).

Highlight 3: Without dynamic profiling, CONDA can reduce the detection latency by around 16× while only incurring 4.65% runtime overhead.

VI. CASE STUDY: COMPATIBILITY WITH PARALLEL HPC SIMULATIONS

In this section, we explore CONDA's compatibility in a real-world HPC simulation that routinely executes in parallel environments. We select SimpleMOC [58], an OpenMP-based 3D neutron transport calculation in water reactor simulation developed by Argonne National Laboratory. The metadata of SimpleMOC can be found in Table II. We compile SimpleMOC using LLVM 15.0, protect it with CONDA, and measure the runtime overheads on an anonymous supercomputer, where each node has an AMD EPYC CPU and 1 TB of memory.

Name	Domain	# Static Instructions	SLOC
SimpleMOC	Water Reactor Simulation	8,300	3,444

TABLE II
METADATA FOR THE REAL-WORLD PARALLEL HPC SIMULATION.

We measure the runtime overheads across different numbers of threads, and these results are presented in Figure 16. As seen, CONDA demonstrates compatibility with multi-threaded programs, and the runtime overhead decreases as the number of threads increases, ranging from 64.23% with 8 threads to 46.23% with 128 threads. We observe the runtime overhead becomes stable after 32 threads, due to the efficient utilization of hardware resources and the strong computation ability in each CPU core. For fault coverage and detection latency, the results in multi-thread scenarios are consistent with what we observed in Section V-C and V-D. Note that no detection techniques can be compatible in such environments while still maintaining the detection effectiveness for holistic datapath errors.

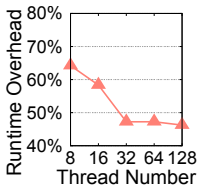


Fig. 16. Runtime overhead of SimpleMOC.

VII. RELATED WORK

A. Data-Flow Error Detection Techniques

Despite expensive hardware solutions such as voltage scaling [59] and lockstep processing [60], [61], software-level solutions for detecting data-flow errors have been extensively studied in the past several decades [9]–[13], [26], [27], [62]–[68]. While algorithm-based fault tolerance is confined to a specific type of applications [64], error detection by duplicating instructions (EDDI) [9] has become increasingly more popular, since it functions at the compiler level and is therefore application-independent, leading many researchers to adopt and build upon it. Didehban et al. [10] identified several non-duplicated instructions at the microarchitecture level and boosted EDDI in ARM processors. With the emerging AI-driven solutions [12], [69]–[71], Laguna et al. [27] utilized a statistic learning-based model to predict instruction vulnerability for selective protection. Mahmoud et al. [13] performed instruction duplication for detecting data-flow errors in NVIDIA GPUs. Those techniques effectively detect data-flow errors; however, they fall short in pinpointing control-flow errors, a significant category of datapath errors.

B. Control-Flow Error Detection Techniques

Software-based control-flow error detection, compared with hardware approaches such as watchdog [72], [73], has become a significant focus in reliability research, receiving more attention than hardware solutions in years times. [15], [18]–[20], [28], [30], [74], [75]. Alkhalifa et al. [75] inherited the architectural portability from assertion checking and proposed an enhanced version for low-overhead and low-latency control-flow protection. Soon after that, Oh et al. [15] opened up a new direction, control-flow error checking by software signatures (CFCSS), to detect control-flow errors by assigning each basic block a unique signature and performing bit-wise error checking. Khudia et al. [18] divided the program into different abstraction levels and performed different magnitudes of signature computation accordingly to reduce the overhead of CFCSS. Zhang et al. [19] simplified the bit-wise operations for error checking in CFCSS partly with accumulating constants for low-cost control-flow error detection. These methods successfully protect programs from control-flow errors.

C. Holistic Detection Techniques for Datapath Errors

Several works also tried to explore holistic protection for datapath errors – transient hardware faults that may manifest control-flow and data-flow distortion [10], [21], [30], [76]. Reis et al. [21] proposed SWIFT, which utilized the functionalities from both EDDI and CFCSS and optimized its performance by reclaiming hardware idle resources, to allow

comprehensive error detection for applications executed for CPU applications. Sharif et al. [30] identified a possibility of checking illegal branch jumps via instruction-level redundancy and boosted an existing data-flow checking technique [10] with control-flow protection. Similarly, Didehban et al. [22] combined a coarse-grained instruction duplication method with an asymmetric software signature for holistic datapath protection against transient hardware faults. However, despite error detection effectiveness, these works exhibit excessive runtime performance penalties. For example, based on the report from [30] and [22], the average runtime overhead of their proposed techniques can be around 178% and 172% on several MiBench programs. Whereas in ConDa, the runtime overhead incurred from generic datapath protection is only 57.79% – this low cost is also the main target of this work.

VIII. Conclusion

In this paper, we propose ConDa, a fast and versatile datapath protection technique with static code transformations and various compiler-level optimizations. Give a source program code, ConDa requires three steps to perform code transformation and generates the protected program code. First, it instruments duplicated instructions and software signatures with lazy code motion mechanism. Then, ConDa performs register-level operations to aggregate checking results across basic blocks. Finally, ConDa places detection branches using adaptive detection placement algorithm through a static loop analysis. A comprehensive evaluation of 38 benchmarks and a real-world parallel HPC simulation demonstrates that ConDa only incurs 57.79% runtime overhead, which is 41.84% faster than baseline methods, with no loss of detection effectiveness in terms of fault coverage and detection latency.

Acknowledgment

This material was based upon work supported by the U.S. Department of Energy, Office of Science, Advanced Scientific Computing Research (ASCR), under contracts DE-AC02-06CH11357, DE-SC0024207, and DE-SC0024559.

References

[1] G. P. Saggese, N. J. Wang, Z. T. Kalbarczyk, S. J. Patel, and R. K. Iyer, "An experimental study of soft errors in microprocessors," *IEEE micro*, vol. 25, no. 6, pp. 30–39, 2005.

[2] D. Tiwari, S. Gupta, J. Rogers, D. Maxwell, P. Rech, S. Vazhkudai, D. Oliveira, D. Londo, N. DeBardeleben, P. Navaux *et al.*, "Understanding gpu errors on large-scale hpc systems and the implications for system design and operation," in *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2015, pp. 331–342.

[3] C. Constantinescu, "Trends and challenges in vlsi circuit reliability," *IEEE micro*, vol. 23, no. 4, pp. 14–19, 2003.

[4] M. Snir, R. W. Wisniewski, J. A. Abraham, S. V. Adve, S. Bagchi, P. Balaji, J. Belak, P. Bose, F. Cappello, B. Carlson *et al.*, "Addressing failures in exascale computing," *The International Journal of High Performance Computing Applications*, vol. 28, no. 2, pp. 129–173, 2014.

[5] P. Shivakumar, M. Kistler, S. W. Keckler, D. Burger, and L. Alvisi, "Modeling the effect of technology trends on the soft error rate of combinational logic," in *Proceedings International Conference on Dependable Systems and Networks*. IEEE, 2002, pp. 389–398.

[6] R. Lucas, J. Ang, K. Bergman, S. Borkar, W. Carlson, L. Carrington, G. Chiu, R. Colwell, W. Dally, J. Dongarra *et al.*, "Doe advanced scientific computing advisory subcommittee (ascac) report: top ten exascale research challenges," USDOE Office of Science (SC)(United States), Tech. Rep., 2014.

[7] A. Marathe, P. E. Bailey, D. K. Lowenthal, B. Rountree, M. Schulz, and B. R. de Supinski, "A run-time system for power-constrained hpc applications," in *High Performance Computing: 30th International Conference, ISC High Performance 2015, Frankfurt, Germany, July 12-16, 2015, Proceedings 30*. Springer, 2015, pp. 394–408.

[8] B. Li, R. Basu Roy, D. Wang, S. Samsi, V. Gadepally, and D. Tiwari, "Toward sustainable hpc: Carbon footprint estimation and environmental implications of hpc systems," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–15.

[9] N. Oh, P. P. Shirvani, and E. J. McCluskey, "Error detection by duplicated instructions in super-scalar processors," *IEEE Transactions on Reliability*, vol. 51, no. 1, pp. 63–75, 2002.

[10] M. Didehban and A. Shrivastava, "nzdc: A compiler technique for near zero silent data corruption," in *Proceedings of the 53rd Annual Design Automation Conference*, 2016, pp. 1–6.

[11] Y. Huang, S. Guo, S. Di, G. Li, and F. Cappello, "Mitigating silent data corruptions in hpc applications across multiple program inputs," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–14.

[12] Q. Lu, G. Li, K. Pattabiraman, M. S. Gupta, and J. A. Rivers, "Configurable detection of sdc-causing errors in programs," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 16, no. 3, pp. 1–25, 2017.

[13] A. Mahmoud, S. K. S. Hari, M. B. Sullivan, T. Tsai, and S. W. Keckler, "Optimizing software-directed instruction replication for gpu error detection," in *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2018, pp. 842–854.

[14] Z. He, Y. Huang, H. Xu, D. Tao, and G. Li, "Demystifying and mitigating cross-layer deficiencies of soft error protection in instruction duplication," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–13.

[15] N. Oh, P. P. Shirvani, and E. J. McCluskey, "Control-flow checking by software signatures," *IEEE transactions on Reliability*, vol. 51, no. 1, pp. 111–122, 2002.

[16] O. Goloubeva, M. Rebaudengo, M. S. Reorda, and M. Violante, "Soft-error detection using control flow assertions," in *Proceedings 18th IEEE Symposium on Defect and Fault Tolerance in VLSI Systems*. IEEE, 2003, pp. 581–588.

[17] R. Vemu and J. Abraham, "Ceda: Control-flow error detection using assertions," *IEEE Transactions on Computers*, vol. 60, no. 9, pp. 1233–1245, 2011.

[18] D. S. Khudia and S. Mahlke, "Low cost control flow protection using abstract control signatures," in *Proceedings of the 14th ACM SIGPLAN/SIGBED conference on Languages, compilers and tools for embedded systems*, 2013, pp. 3–12.

[19] Z. Zhang, S. Park, and S. Mahlke, "Path sensitive signatures for control flow error detection," in *The 21st ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems*, 2020, pp. 62–73.

[20] A. Rhisheekesan, R. Jeyapaul, and A. Shrivastava, "Control flow checking or not?(for soft errors)," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 18, no. 1, pp. 1–25, 2019.

[21] G. A. Reis, J. Chang, N. Vachharajani, R. Rangan, and D. I. August, "Swift: Software implemented fault tolerance," in *International symposium on Code generation and optimization*. IEEE, 2005, pp. 243–254.

[22] M. Didehban, H. So, P. Gali, A. Shrivastava, and K. Lee, "Generic soft error data and control flow error detection by instruction duplication," *IEEE Transactions on Dependable and Secure Computing*, 2023.

[23] S. Wang, G. Zhang, J. Wei, Y. Wang, J. Wu, and Q. Luo, "Understanding silent data corruptions in a large production cpu population," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 216–230.

[24] P. H. Hochschild, P. Turner, J. C. Mogul, R. Govindaraju, P. Ranganathan, D. E. Culler, and A. Vahdat, "Cores that don't count," in *Proceedings of the Workshop on Hot Topics in Operating Systems*, 2021, pp. 9–16.

[25] H. D. Dixit, S. Pendharkar, M. Beadon, C. Mason, T. Chakravarthy, B. Muthiah, and S. Sankar, "Silent data corruptions at scale," *arXiv preprint arXiv:2102.11245*, 2021.

[26] C. Kalra, F. Previlon, N. Rubin, and D. Kaeli, "Armorall: Compiler-based resilience targeting gpu applications," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 17, no. 2, pp. 1–24, 2020.

[27] I. Laguna, M. Schulz, D. F. Richards, J. Calhoun, and L. Olson, "Ipas: Intelligent protection against silent output corruption in scientific applications," in *Proceedings of the 2016 International Symposium on Code Generation and Optimization*, 2016, pp. 227–238.

[28] S. Schuster, P. Ulbrich, I. Stilkerich, C. Dietrich, and W. Schröder-Preikschat, "Demystifying soft-error mitigation by control-flow checking–a new perspective on its effectiveness," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 16, no. 5s, pp. 1–19, 2017.

[29] R. W. Hamming, "Error detecting and error correcting codes," *The Bell system technical journal*, vol. 29, no. 2, pp. 147–160, 1950.

[30] U. Sharif, D. Mueller-Gritschneder, and U. Schlichtmann, "Repair: Control flow protection based on register pairing updates for sw-implemented hw fault tolerance," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 20, no. 5s, pp. 1–22, 2021.

[31] J. A. Stratton, C. Rodrigues, I.-J. Sung, N. Obeid, L.-W. Chang, N. Anssari, G. D. Liu, and W.-m. W. Hwu, "Parboil: A revised benchmark suite for scientific and commercial throughput computing," *Center for Reliable and High-Performance Computing*, vol. 127, p. 27, 2012.

[32] Y. Huang, Z. He, L. Li, and G. Li, "Characterizing runtime performance variation in error detection by duplicating instructions," in *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2023, pp. 730–741.

[33] S. Rixner, W. J. Dally, U. J. Kapasi, P. Mattson, and J. D. Owens, "Memory access scheduling," *ACM SIGARCH Computer Architecture News*, vol. 28, no. 2, pp. 128–138, 2000.

[34] S. Di, M. S. Bouguerra, L. Bautista-Gomez, and F. Cappello, "Optimization of multi-level checkpoint model for large scale hpc applications," in *2014 IEEE 28th international parallel and distributed processing symposium*. IEEE, 2014, pp. 1181–1190.

[35] M. A. Heroux, D. W. Doerfler, P. S. Crozier, J. M. Willenbring, H. C. Edwards, A. Williams, M. Rajan, E. R. Keiter, H. K. Thornquist, and R. W. Numrich, "Improving Performance via Mini-applications," Sandia National Laboratories, Tech. Rep. SAND2009-5574, 2009.

[36] J. R. Tramm, A. R. Siegel, T. Islam, and M. Schulz, "XSBench - the development and verification of a performance abstraction for Monte Carlo reactor analysis," in *PHYSOR 2014 - The Role of Reactor Physics toward a Sustainable Future*, Kyoto, 2014. [Online]. Available: https://www.mcs.anl.gov/papers/P5064-0114.pdf

[37] N. P. Benchmarks, "Nas parallel benchmarks," *CG and IS*, 2006.

[38] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown, "Mibench: A free, commercially representative embedded benchmark suite," in *Proceedings of the fourth annual IEEE international workshop on workload characterization. WWC-4 (Cat. No. 01EX538)*. IEEE, 2001, pp. 3–14.

[39] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *2009 IEEE international symposium on workload characterization (IISWC)*. Ieee, 2009, pp. 44–54.

[40] S. C. Woo, M. Ohara, E. Torrie, J. P. Singh, and A. Gupta, "The splash-2 programs: Characterization and methodological considerations," *ACM SIGARCH computer architecture news*, vol. 23, no. 2, pp. 24–36, 1995.

[41] Z. Li, H. Menon, K. Mohror, P.-T. Bremer, Y. Livant, and V. Pascucci, "Understanding a program's resiliency through error propagation," in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 362–373.

[42] M. Ebrahimi, M. Rashvand, F. Kaddachi, M. B. Tahoori, and G. Di Natale, "Revisiting software-based soft error mitigation techniques via accurate error generation and propagation models," in *2016 IEEE 22nd International Symposium on On-Line Testing and Robust System Design (IOLTS)*. IEEE, 2016, pp. 66–71.

[43] Q. Lu, M. Farahani, J. Wei, A. Thomas, and K. Pattabiraman, "Llfi: An intermediate code-level fault injection tool for hardware faults," in *2015 IEEE International Conference on Software Quality, Reliability and Security*. IEEE, 2015, pp. 11–16.

[44] M. Didehban, S. R. D. Lokam, and A. Shrivastava, "Incheck: An in-application recovery scheme for soft errors," in *Proceedings of the 54th Annual Design Automation Conference 2017*, 2017, pp. 1–6.

[45] P. R. Bodmann, G. Papadimitriou, R. L. R. Junior, D. Gizopoulos, and P. Rech, "Soft error effects on arm microprocessors: Early estimations versus chip measurements," *IEEE Transactions on Computers*, vol. 71, no. 10, pp. 2358–2369, 2021.

[46] Q. Guan, N. Debardeleben, S. Blanchard, and S. Fu, "F-sefi: A fine-grained soft error fault injection tool for profiling application vulnerability," in *2014 IEEE 28th International Parallel and Distributed Processing Symposium*. IEEE, 2014, pp. 1245–1254.

[47] Uiuc openimpact effort, "the openimpact ia-64 compiler." [deprecated]. [Online]. Available: http://gelato.uiuc.edu/

[48] Llvm-or1k compiler [open access]. [Online]. Available: https://github.com/openrisc/llvm-or1k

[49] L. Palazzi, G. Li, B. Fang, and K. Pattabiraman, "A tale of two injectors: End-to-end comparison of ir-level and assembly-level fault injection," in *2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2019, pp. 151–162.

[50] "Intel pin," https://www.intel.com/content/www/us/en/developer/articles/tool/pin-a-dynamic-binary-instrumentation-tool.html.

[51] J. Calhoun, L. Olson, and M. Snir, "Flipit: An llvm based fault injector for hpc," in *Euro-Par 2014: Parallel Processing Workshops: Euro-Par 2014 International Workshops, Porto, Portugal, August 25-26, 2014, Revised Selected Papers, Part I 20*. Springer, 2014, pp. 547–558.

[52] S. Vishal, C. Sharma, and G. Gopalakrishnan, "Towards re-seiliency evaluation of vector programs," in *21st IEEE Workshop on Dependable Parallel, Distributed and Network-Centric Systems (DPDNS)*, 2016.

[53] D. Li, J. S. Vetter, and W. Yu, "Classifying soft error vulnerabilities in extreme-scale scientific applications using a binary instrumentation tool," in *SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE, 2012, pp. 1–11.

[54] U. Wappler and C. Fetzer, "Hardware fault injection using dynamic binary instrumentation: Fitgrind," *Proceedings Supplemental Volume of EDCC-6*, 2006.

[55] S. K. S. Hari, S. V. Adve, H. Naeimi, and P. Ramachandran, "Relyzer: Exploiting application-level fault equivalence to analyze application resiliency to transient faults," *ACM SIGARCH Computer Architecture News*, vol. 40, no. 1, pp. 123–134, 2012.

[56] S. Feng, S. Gupta, A. Ansari, and S. Mahlke, "Shoestring: probabilistic soft error reliability on the cheap," *ACM SIGARCH Computer Architecture News*, vol. 38, no. 1, pp. 385–396, 2010.

[57] G. Li, K. Pattabiraman, S. K. S. Hari, M. Sullivan, and T. Tsai, "Modeling soft-error propagation in programs," in *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2018, pp. 27–38.

[58] G. Gunow, J. Tramm, B. Forget, K. Smith, and T. He, "SimpleMOC – a performance abstraction for 3D MOC," in *ANS & M&C 2015 - Joint International Conference on Mathematics and Computation (M&C), Supercomputing in Nuclear Applications (SNA) and the Monte Carlo (MC) Method*, 2015.

[59] K.-C. Wu and D. Marculescu, "Power-aware soft error hardening via selective voltage scaling," in *2008 IEEE International Conference on Computer Design*. IEEE, 2008, pp. 301–306.

[60] S. Poledna, *Fault-tolerant real-time systems: The problem of replica determinism*. Springer Science & Business Media, 2007, vol. 345.

[61] E. Ozer, B. Venu, X. Iturbe, S. Das, S. Lyberis, J. Biggs, P. Harrod, and J. Penton, "Error correlation prediction in lockstep processors for safety-critical systems," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 737–748.

[62] Y. Huang, S. Guo, S. Di, G. Li, and F. Cappello, "Hardening selective protection across multiple program inputs for hpc applications," in *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2022, pp. 437–438.

[63] J. Chang, G. A. Reis, and D. I. August, "Automatic instruction-level software-only recovery," in *International Conference on Dependable Systems and Networks (DSN'06)*. IEEE, 2006, pp. 83–92.

[64] K. Zhao, S. Di, S. Li, X. Liang, Y. Zhai, J. Chen, K. Ouyang, F. Cappello, and Z. Chen, "Ft-cnn: Algorithm-based fault tolerance for convolutional neural networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 7, pp. 1677–1689, 2020.

[65] M. H. Rahman, A. Shamji, S. Guo, and G. Li, "Peppa-x: finding program test inputs to bound silent data corruption vulnerability in hpc applications," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–13.

[66] M. H. Rahman, S. Di, S. Guo, X. Lu, G. Li, and F. Cappello, "Druto: Upper-bounding silent data corruption vulnerability in gpu applications," in *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2024, pp. 582–594.

[67] M. H. Rahman, S. Laskar, and G. Li, "Investigating the impact of transient hardware faults on deep learning neural network inference," *Software Testing, Verification and Reliability*, p. e1873, 2024.

[68] B. Zhang, L. Yang, G. Li, and H. Xu, "Investigating the impact of high-level software design on low-level hardware fault resilience," in *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks-Supplemental Volume (DSN-S)*. IEEE, 2023, pp. 163–167.

[69] H. Yue, X. Wei, G. Li, J. Zhao, N. Jiang, and J. Tan, "G-sepm: building an accurate and efficient soft error prediction model for gpgpus," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–15.

[70] B. Zhang, Y. Huang, and G. Li, "Salus: A novel data-driven monitor that enables real-time safety in autonomous driving systems," in *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 2022, pp. 85–94.

[71] Z. Chen, T. Verrecchia, H. Sun, J. Booth, and P. Raghavan, "Dynamic selective protection of sparse iterative solvers via ml prediction of soft error impacts," in *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*, 2023, pp. 488–491.

[72] N. Murphy and M. Barr, "Watchdog timers," *Embedded Systems Programming*, vol. 14, no. 11, pp. 79–80, 2001.

[73] N. R. Saxena and E. J. McCluskey, "Control-flow checking using watchdog assists and extended-precision checksums," *IEEE Transactions on Computers*, vol. 39, no. 4, pp. 554–559, 1990.

[74] E. Borin, C. Wang, Y. Wu, and G. Araujo, "Software-based transparent and comprehensive control-flow error detection," in *International Symposium on Code Generation and Optimization (CGO'06)*. IEEE, 2006, pp. 13–pp.

[75] Z. Alkhalifa, V. S. Nair, N. Krishnamurthy, and J. A. Abraham, "Design and evaluation of system-level checks for on-line control flow error detection," *IEEE Transactions on Parallel and Distributed Systems*, vol. 10, no. 6, pp. 627–641, 1999.

[76] E. Chielle, F. Rosa, G. S. Rodrigues, L. A. Tambara, J. Tonfat, E. Macchione, F. Aguirre, N. Added, N. Medina, V. Aguiar *et al.*, "Reliability on arm processors against soft errors through sihft techniques," *IEEE Transactions on Nuclear Science*, vol. 63, no. 4, pp. 2208–2216, 2016.